

L4-P04 练习手册

模型测试和评估

概述

本手册包含三个递进式实践，完整覆盖“训练后拥有一个可用模型”的全周期流程：

- 实践 1：将 LoRA Adapter 与基座模型合并，得到独立可部署的完整模型
- 实践 2：对合并模型执行批量推理，生成预测结果
- 实践 3：计算 BLEU、ROUGE、BERTScore 等多维计量指标，实现完整自动化评估

实践 1：合并 LoRA Adapter 与基座模型

程序文件：`ex1_merge_lora_model.py`

1.1 背景知识

LoRA 训练完成后会生成一个第一次“Adapter”目录，其中只包含增量权重矩阵（A、B）而非完整模型。在实际部署时，通常需要将 LoRA 权重合并进基座模型，以便：

- 无需额外加载 PEFT 库即可直接推理
- 减少推理期的 Adapter 计算开销
- 模型格式层级与标准预训练模型完全一致，便于分发和部署

1.2 核心函数：merge_lora_and_base

该函数分四步完成 LoRA 与基座模型的合并：

1. 加载基座模型

从 `base_model_path` 目录加载原始大语言模型（如 Qwen1.5-1.8B-Chat），并配置对应的 Tokenizer。

2. 加载 LoRA Adapter

从 `lora_model_path` 目录读入 LoRA 增量权重，通过 PEFT 将其挂载到基座模型上。

3. 合并权重（merge_and_unload）

调用 `merge_and_unload()`：将 LoRA 增量矩阵（ $\alpha/r \cdot B \cdot A$ ）加回基座路径，同时移除 LoRA 结构。得到的模型格式与普通预训练模型完全一致。

4. 保存合并模型

将合并后的完整模型以及对应 Tokenizer 均保存到 `save_path` 目录。

1.3 路径配置

```
# 基座模型路径（原始预训练权重）
base_model_path = "../models/Qwen1.5-1.8B-Chat"

# LoRA Adapter 路径（微调训练得到的增量权重）
lora_model_path = "./lora-adapter"

# 合并后模型的输出路径
save_path = "./qwen-merged-model"
```

注意事项

合并操作会占用较大内存，建议在 GPU 环境下执行。

合并后的模型不再依赖 PEFT 库，可直接用 Transformers 加载推理。

实践 2：测试合并模型推理

程序文件：`ex2_test_llm_model.py`

2.1 背景说明

模型合并完成后，需要对其推理效果进行系统性验证。本实践以批量方式从 JSONL 文件中读取输入，对每条进行模型推理，并将预测结果与参考答案分别写入文件，为后续自动化评估（实践 3）做准备。

2.2 程序流程

5. 加载模型与分词器

- 使用 `AutoTokenizer + AutoModelForCausalLM` 从 `./merged_model` 目录加载模型。
- `torch_dtype=torch.float16` 节省显存；`device_map="auto"` 自动分配 GPU/CPU。

6. 指定输入输出文件

- `test_data.jsonl`——JSONL 格式输入文件，每行 `{"input": "...", "reference": "..."}`
- `predictions.txt`——模型生成结果，每行一条
- `references.txt`——原始参考答案，每行一条

7. 推理函数 `generate_response`

- 将 `user_input` 封装为 ChatML 对话格式（`system + user`）。
- 调用 `tokenizer.apply_chat_template` 构建输入 token。
- 调用 `model.generate` 生成文本；`tokenizer.decode` 转为可读字符串。

8. 逐条处理并保存结果

- 依次读取 `test_data.jsonl` 的每一行。

- 调用 `generate_response` 得到预测结果并分别写入两个输出文件。

2.3 文件格式说明

输入文件: `test_data.jsonl`

```
{ "input": "患者：我这两天头痛，还有点恶心，需要去医院吗？", "reference": "建议尽快去医院检查，可能是偏头痛或感染" }
{ "input": "翻译：人工智能正在改变世界。", "reference": "Artificial intelligence is changing the world." }
{ "input": "请解释：什么是机器学习？", "reference": "机器学习是一种利用数据训练算法，使其自动改进性能的技术。" }
```

输出对比示例

输入	模型预测 (predictions.txt)	参考答案 (references.txt)
患者：我这两天头痛.....	建议您尽快去医院检查，以排除严重疾病。	建议尽快去医院检查，可能是偏头痛或感染
翻译：人工智能正在改变世界。	Artificial intelligence is changing the world.	Artificial intelligence is changing the world.
请解释：什么是机器学习？	机器学习是一种通过数据训练模型来改进性能的方法。	机器学习是一种利用数据训练算法，使其自动改进性能的技术。

实践 3：计算自动化评估指标

程序文件: `ex3_llm_metrics_suite_cli_table.py`

3.1 背景说明

自动化指标评估是模型训练后不可缺少的环节。本实践集成 BLEU、ROUGE、BERTScore、Accuracy、F1、Exact Match 等完整指标套件，并以表格形式输出，方便对比多个模型或训练轮次的表现。

3.2 指标说明

指标	说明
BLEU	基于 n-gram 精确率评估生成文本与参考文本的相似度，常用于翻译质量评估
ROUGE-1 / ROUGE-2 / ROUGE-L	基于 n-gram 召回率衡量生成摘要与参考摘要的重叠程度
BERTScore-P / R / F1	使用 BERT 语义嵌入，衡量生成文本与参考文本在语义层面的精确率、召回率和 F1

Accuracy	分类任务中，预测正确的样本占总样本的比例
F1 (macro)	各类别 F1 分数的宏平均，适合类别不平衡场景
Exact Match	预测结果与参考答案完全一致的比例

3.3 安装依赖

```
pip install datasets evaluate scikit-learn tabulate sacrebleu rouge_score
```

以下为程序成功运行的验证版本：

```
datasets==3.4.1
evaluate==0.4.6
sacrebleu==2.5.1
scikit-learn==1.7.2
tabulate==0.9.0
```

3.4 运行命令

```
# 评估第一组数据
python ex3_llm_metrics_suite_cli_table.py \
  --pred ex3_predictions.txt \
  --ref ex3_references.txt \
  --lang zh

# 评估第二组数据
python ex3_llm_metrics_suite_cli_table.py \
  --pred ex3_predictions_2.txt \
  --ref ex3_references_2.txt \
  --lang zh
```

3.5 输出示例

程序如正常运行，会以表格形式输出如下评估结果（以下数据仅供参考）：

指标	分数（示例）
BLEU	0.5123
ROUGE-1	0.6012
ROUGE-2	0.4331
ROUGE-L	0.5888
BERTScore-P	0.8124
BERTScore-R	0.8059
BERTScore-F1	0.8091
Accuracy	0.7500
F1 (macro)	0.7400

Exact Match	0.7000
-------------	--------

实践此要

实践 1 得到合并模型 → 实践 2 生成预测 / 参考文件 → 实践 3 对评估结果量化
三个实践前后衔接，共同构成一个不依赖 PEFT 的自包含模型评估全流程。